

UNIT 4

● What is Dependency Injection?

Dependency Injection, or DI, is a software design technique in which a class receives the objects it needs from an external source instead of creating those objects by itself.

*The objects required by a class are called **dependencies**.*

For example, if a Notification class needs an email service, the email service is its dependency.

● Without Dependency Injection

using System;

```
public class EmailService
{
    public void SendEmail()
    {
        Console.WriteLine("Email sent.");
    }
}

public class Notification
{
    private readonly EmailService _emailService
        = new EmailService();

    public void Notify()
    {
        _emailService.SendEmail();
    }
}

public class Program
{
    public static void Main()
    {
        Notification notification = new Notification();

        notification.Notify();
    }
}
```

Output

Email sent.

● Why This Is Tight Coupling

The important line is:

```
private readonly EmailService _emailService
    = new EmailService();
```

The Notification class directly creates the EmailService object.

Therefore:

- Notification is directly dependent on EmailService.
- **EmailService cannot easily be replaced with SmsService.**

Example:

Replacing EmailService with SmsService

Suppose we create an SMS service:

```
public class SmsService
{
    public void SendSms()
    {
        Console.WriteLine("SMS sent.");
    }
}
```

The existing Notification class cannot use it directly because it expects an EmailService object:

private readonly EmailService emailService = new EmailService();

To use SMS, we must change the Notification class:

```
public class Notification
{
    private readonly SmsService _smsService
        = new SmsService();

    public void Notify()
    {
        _smsService.SendSms();
    }
}
```

● What had to change?

Three parts were modified:

EmailService

was changed to:

SmsService

The field name changed:

emailService

to:

smsService

The method call changed:

SendEmail()

to:

SendSms()

Key Point

To replace the service, we had to edit the Notification class itself.

That is why replacement is difficult.

- Changing the implementation requires changing Notification.

This is not Dependency Injection.

So we need : ***Dependency Injection***

DI may also be managed in two broader ways:

1. Manual Dependency Injection, where objects are created and passed manually.

2. DI Container-based implementation, using tools such as Microsoft Dependency Injection or Autofac.

1. Dependency Injection: Manual Constructor Injection

◆ Major Points

- **IMessageService** defines the behaviour required by the dependent class.
- **EmailService** implements that behaviour.
- **Notification** depends on the interface instead of the concrete class.
- The dependency is created outside and passed through the constructor.

✓ How to Achieve It

1. Create an interface for the required service.
2. Create a concrete class that implements the interface.
3. Receive the interface through the dependent class constructor.
4. Create the implementation outside the dependent class.
5. Pass the implementation to the constructor and call the required method.

Complete Runnable Program

```
using System;

public interface IMessageService
{
    void SendMessage();
}

public class EmailService : IMessageService
{
    public void SendMessage()
    {
        Console.WriteLine("Email sent successfully.");
    }
}

public class Notification
{
    // KEY: Depend on the interface, not EmailService directly.
    private readonly IMessageService _messageService;

    // KEY: Receive the dependency through the constructor.
    public Notification(IMessageService messageService)
    {
        _messageService = messageService;
    }

    public void Notify()
    {
        // KEY: Use the injected dependency.
    }
}
```

```

    _messageService.SendMessage();
}
}

public static class Program
{
    public static void Main()
    {
        // KEY: Create the dependency outside Notification.
        IMessageService service = new EmailService();

        // KEY: Inject the dependency into Notification.
        var notification = new Notification(service);

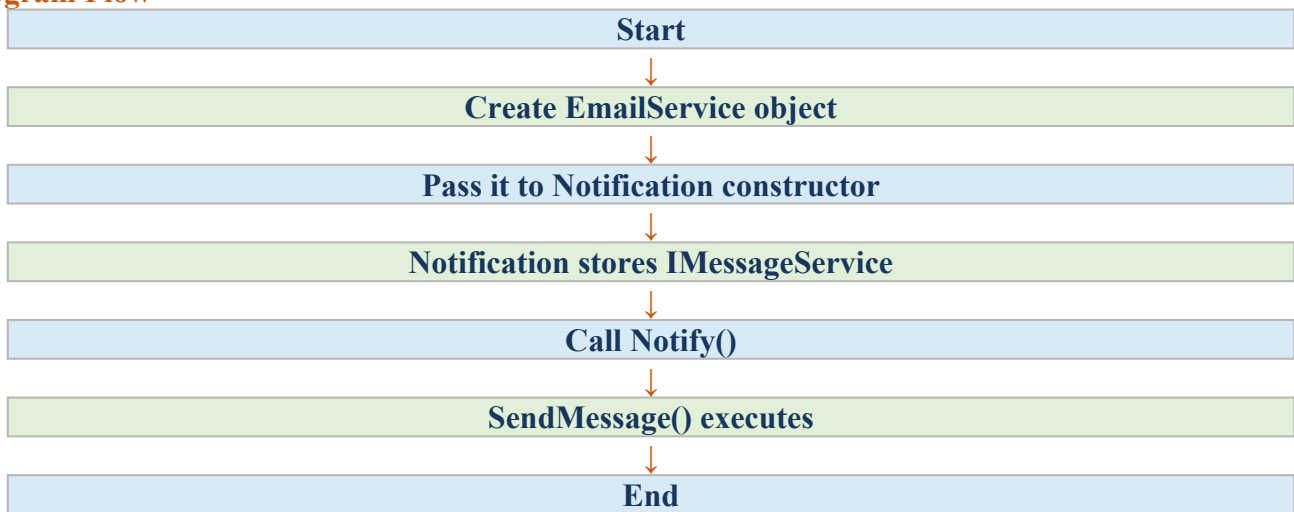
        notification.Notify();
    }
}

```

► Expected Output

- Email sent successfully.

Program Flow



Note point: Meaning of Each Keyword

private readonly IMessageService _messageService;

● private

The field can be accessed **only inside the same class**.

readonly

The field can be assigned:

- When it is declared, or
- Inside the class constructor

After that, it cannot be assigned a different object.

IMessageService

This is the **data type** of the field.

It is an interface that defines the message-service operations.

_messageService

This is the **name of the private field**.

The underscore `_` is a common C# naming convention for private fields.

✓ **Complete Meaning**

This line declares a **private, read-only field named `_messageService` that can store an object implementing `IMessageService`**

2. Dependency Injection Using Microsoft DI Container

• Short Note

- Microsoft provides a built-in DI framework through **`Microsoft.Extensions.DependencyInjection`**.
- **`ServiceCollection`** stores registrations.
- **`ServiceProvider`** creates and supplies registered objects.
- The container automatically examines constructors and injects required dependencies.

✓ How to Achieve It

1. Create a console project.
2. Install `Microsoft.Extensions.DependencyInjection`.
3. Create interfaces and implementations.
4. Register services with `ServiceCollection`.
5. Build the `ServiceProvider`.
6. Resolve the main class and execute the method.

Project Commands

```
dotnet new console -n DependencyInjectionDemo
cd DependencyInjectionDemo
dotnet add package Microsoft.Extensions.DependencyInjection
dotnet run
```

Complete Program.cs

```
using System;
using Microsoft.Extensions.DependencyInjection;

public interface IHome
{
    void ProvideShelter();
}

public interface ISchool
{
    void ProvideEducation();
}

public interface IHospital
{
    void ProvideTreatment();
}

public class Home : IHome
{
    public void ProvideShelter() =>
```

```

    Console.WriteLine("Home provides shelter.");
}

public class School : ISchool
{
    public void ProvideEducation() =>
        Console.WriteLine("School provides education.");
}

public class Hospital : IHospital
{
    public void ProvideTreatment() =>
        Console.WriteLine("Hospital provides treatment.");
}

public class Person
{
    private readonly IHome _home;
    private readonly ISchool _school;
    private readonly IHospital _hospital;

    // KEY: The container will supply all three dependencies.
    public Person(IHome home, ISchool school, IHospital hospital)
    {
        _home = home;
        _school = school;
        _hospital = hospital;
    }

    public void PerformActivities()
    {
        _home.ProvideShelter();
        _school.ProvideEducation();
        _hospital.ProvideTreatment();
    }
}

public static class Program
{
    public static void Main()
    {
        var services = new ServiceCollection();

        // KEY: Register interface-to-implementation mappings.
        services.AddTransient<IHome, Home>();
        services.AddTransient<ISchool, School>();
        services.AddTransient<IHospital, Hospital>();

        // KEY: Register the main class.
        services.AddTransient<Person>();
    }
}

```

```
// KEY: Build the DI container.
using ServiceProvider provider = services.BuildServiceProvider();

// KEY: Resolve Person; dependencies are injected automatically.
Person person = provider.GetRequiredService<Person>();

person.PerformActivities();
}
}
```

► Expected Output

- Home provides shelter.
- School provides education.
- Hospital provides treatment.

Program Flow



*The following is a **complete runnable MVC structure** based on your original Home, School, Hospital, and Person example.*

● 1. Create the MVC Project

Open Command Prompt or Terminal:

```
dotnet new mvc -n DependencyInjectionMvc
```

```
cd DependencyInjectionMvc
```

ASP.NET Core MVC uses `AddControllersWithViews()` to register MVC controllers, Views, Razor support, model validation, and related services.

2. Project Structure

```
DependencyInjectionMvc/
```

```
|
|— Controllers/
|   |— HomeController.cs
```

```

|
├── Interfaces/
│   ├── IHome.cs
│   ├── ISchool.cs
│   └── IHospital.cs
├── Models/
│   └── ActivityViewModel.cs
├── Services/
│   ├── Home.cs
│   ├── School.cs
│   ├── Hospital.cs
│   └── Person.cs
├── Views/
│   ├── Home/
│   │   └── Index.cshtml
│   ├── Shared/
│   │   └── _Layout.cshtml
│   ├── _ViewImports.cshtml
│   └── _ViewStart.cshtml
├── wwwroot/
├── Program.cs
└── DependencyInjectionMvc.csproj

```

● 3. Interfaces

Interfaces/IHome.cs

```
namespace DependencyInjectionMvc.Interfaces;
```

```
public interface IHome
{
    string ProvideShelter();
}
```

Key point

```
string ProvideShelter();
```

This method defines the operation that every home implementation must provide.

Interfaces/ISchool.cs

```
namespace DependencyInjectionMvc.Interfaces;
```

```
public interface ISchool
{
    string ProvideEducation();
}
```

Interfaces/IHospital.cs

```
namespace DependencyInjectionMvc.Interfaces;
```

```
public interface IHospital
{
    string ProvideTreatment();
}
```

Why return string?

In the original console application, the methods used:

```
Console.WriteLine();
```

In MVC, the result must normally be sent to a View and displayed in the browser. Therefore, these methods return strings instead of writing directly to the console.

● 4. Service Implementations

Services/Home.cs

```
using DependencyInjectionMvc.Interfaces;
```

```
namespace DependencyInjectionMvc.Services;
```

```
public class Home : IHome
{
    public string ProvideShelter()
    {
        return "Home provides shelter.";
    }
}
```

Major point

```
public class Home : IHome
```

This means that Home implements the IHome interface.

Services/School.cs

```
using DependencyInjectionMvc.Interfaces;
```

```
namespace DependencyInjectionMvc.Services;
```

```
public class School : ISchool
{
    public string ProvideEducation()
    {
```

```

        return "School provides education.";
    }
}

```

Services/Hospital.cs

```

using DependencyInjectionMvc.Interfaces;

namespace DependencyInjectionMvc.Services;

public class Hospital : IHospital
{
    public string ProvideTreatment()
    {
        return "Hospital provides treatment.";
    }
}

```

5. Person Service

Services/Person.cs

```

using DependencyInjectionMvc.Interfaces;
using DependencyInjectionMvc.Models;

namespace DependencyInjectionMvc.Services;

public class Person
{
    private readonly IHome _home;
    private readonly ISchool _school;
    private readonly IHospital _hospital;

    // KEY: ASP.NET Core supplies all three dependencies.
    public Person(
        IHome home,
        ISchool school,
        IHospital hospital)
    {
        _home = home;
        _school = school;
        _hospital = hospital;
    }

    public ActivityViewModel PerformActivities()
    {
        return new ActivityViewModel

```

```

    {
        ShelterMessage = _home.ProvideShelter(),
        EducationMessage = _school.ProvideEducation(),
        TreatmentMessage = _hospital.ProvideTreatment()
    };
}
}

```

Important program section

```

public Person(
    IHome home,
    ISchool school,
    IHospital hospital)

```

This is **Constructor Injection**.

ASP.NET Core examines the constructor and supplies registered implementations for:

- IHome
- ISchool
- IHospital

Field meanings

```
private readonly IHome _home;
```

- private: accessible only inside Person
- readonly: assigned during declaration or inside the constructor
- IHome: interface type
- _home: private field name

● 6. View Model

Models/ActivityViewModel.cs

```
namespace DependencyInjectionMvc.Models;
```

```

public class ActivityViewModel
{
    public string ShelterMessage { get; set; } = string.Empty;

    public string EducationMessage { get; set; } = string.Empty;

    public string TreatmentMessage { get; set; } = string.Empty;
}

```

Purpose

The View Model transfers the results from the Person service to the Razor View.

Person service



ActivityViewModel



HomeController



● 7. MVC Controller

Controllers/HomeController.cs

Replace the existing HomeController.cs with:

```
using DependencyInjectionMvc.Models;
using DependencyInjectionMvc.Services;
using Microsoft.AspNetCore.Mvc;

namespace DependencyInjectionMvc.Controllers;

public class HomeController : Controller
{
    private readonly Person _person;

    // KEY: Person is injected into the controller.
    public HomeController(Person person)
    {
        _person = person;
    }

    public IActionResult Index()
    {
        ActivityViewModel model =
            _person.PerformActivities();

        return View(model);
    }
}
```

Major program points

public HomeController(Person person)

ASP.NET Core injects the registered Person service into the controller.

ActivityViewModel model =

_person.PerformActivities();

The controller calls the business operation.

return View(model);

The result is sent to the Razor View.

ASP.NET Core MVC controllers explicitly request dependencies through constructors, and the runtime resolves those services from the built-in service container.

● 8. Razor View

Views/Home/Index.cshtml

Replace the existing file with:

@model DependencyInjectionMvc.Models.ActivityViewModel

```
@{
    ViewData["Title"] = "Dependency Injection Example";
}
```

```
<div class="container mt-5">
    <div class="text-center">
        <h1 class="text-primary">
            Dependency Injection in ASP.NET Core MVC
        </h1>

        <p class="lead">
            Home, School, and Hospital services were injected
            into the Person class.
        </p>
    </div>
```

```
<div class="card border-primary mb-3">
    <div class="card-header bg-primary text-white">
        Home Service
    </div>
```

```
<div class="card-body">
    <p class="card-text">
        @Model.ShelterMessage
    </p>
</div>
</div>
```

```
<div class="card border-success mb-3">
    <div class="card-header bg-success text-white">
        School Service
    </div>
```

```
<div class="card-body">
    <p class="card-text">
        @Model.EducationMessage
    </p>
</div>
</div>
```

```
<div class="card border-danger mb-3">
    <div class="card-header bg-danger text-white">
        Hospital Service
```

```

</div>

<div class="card-body">
  <p class="card-text">
    @Model.TreatmentMessage
  </p>
</div>
</div>
</div>

```

Major View points

@model DependencyInjectionMvc.Models.ActivityViewModel

Declares the model type received by the View.

@Model.ShelterMessage

Displays the result returned by the Home service.

@Model.EducationMessage

Displays the result returned by the School service.

@Model.TreatmentMessage

Displays the result returned by the Hospital service.

● 9. Complete Program.cs

Program.cs

```
using DependencyInjectionMvc.Interfaces;
```

```
using DependencyInjectionMvc.Services;
```

```
var builder = WebApplication.CreateBuilder(args);
```

```
// KEY 1: Register MVC controllers and Views.
```

```
builder.Services.AddControllersWithViews();
```

```
// KEY 2: Register interface-to-implementation mappings.
```

```
builder.Services.AddTransient<IHome, Home>();
```

```
builder.Services.AddTransient<ISchool, School>();
```

```
builder.Services.AddTransient<IHospital, Hospital>();
```

```
// KEY 3: Register the Person service.
```

```
builder.Services.AddTransient<Person>();
```

```
var app = builder.Build();
```

```
// Configure the HTTP request pipeline.
```

```
if (!app.Environment.IsDevelopment())
```

```
{
```

```
    app.UseExceptionHandler("/Home/Error");
```

```
    app.UseHsts();
```

```
}

app.UseHttpsRedirection();
app.UseStaticFiles();

app.UseRouting();

app.UseAuthorization();

// KEY 4: Configure the default MVC route.
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

Important Program.cs Lines

Register MVC

```
builder.Services.AddControllersWithViews();
```

This enables:

- MVC controllers
- Razor Views
- Model binding
- Model validation
- Tag Helpers
- Razor View Engine

Register IHome

```
builder.Services.AddTransient<IHome, Home>();
```

Meaning:

Whenever an IHome service is required, create and provide a Home object.

Register ISchool

```
builder.Services.AddTransient<ISchool, School>();
```

Meaning:

Whenever ISchool is required, provide a School object.

Register IHospital

```
builder.Services.AddTransient<IHospital, Hospital>();
```

Meaning:

Whenever IHospital is required, provide a Hospital object.

Register Person

```
builder.Services.AddTransient<Person>();
```

Meaning:

Allow ASP.NET Core to create and inject the Person service.

10. How ASP.NET Core Creates the Objects

When the browser requests the home page, ASP.NET Core needs to create:

HomeController

The controller constructor requires:

Person

The Person constructor requires:

IHome

ISchool

IHospital

ASP.NET Core checks the service registrations and conceptually creates:

```
var controller = new HomeController(  
    new Person(  
        new Home(),  
        new School(),  
        new Hospital()  
    )  
);
```

You do not write this object-creation code manually. The built-in DI container performs it automatically.

11. Complete Dependency Flow

Browser requests /



MVC routing selects HomeController



ASP.NET Core attempts to create HomeController



HomeController requires Person



ASP.NET Core attempts to create Person



Person requires IHome, ISchool, and IHospital



DI container checks service registrations



IHome → Home

ISchool → School

IHospital → Hospital



Container creates Home, School, and Hospital



Dependencies are injected into Person



Person is injected into HomeController



Index() action executes



Person.PerformActivities() executes



ActivityViewModel is created



Controller passes model to Index.cshtml



Razor View generates HTML



Browser displays all three messages

12. Run the Application

Use:

dotnet restore

dotnet build

dotnet run

Open the address displayed in the terminal, commonly similar to:

https://localhost:7000

The exact port may be different on your computer.

13. Expected Browser Output

Dependency Injection in ASP.NET Core MVC

Home Service

Home provides shelter.

School Service

School provides education.

Hospital Service

Hospital provides treatment.

● 14. Console Application vs MVC Application

Console application	ASP.NET Core MVC
Uses ServiceCollection manually	Uses builder.Services
Manually builds ServiceProvider	Framework builds the container
Manually resolves Person	Controller receives Person automatically
Uses Console.WriteLine()	Sends data to a Razor View
Starts from Main() logic	Starts through the HTTP request pipeline

Displays output in console	Displays output in browser
----------------------------	----------------------------

[Extra contents-not necessary]3. Dependency Injection Using Autofac

• Short Note

- **Autofac** is a third-party IoC container for .NET.
- It uses **ContainerBuilder** for registration.
- It creates a container and resolves services through a lifetime scope.
- Autofac is useful when advanced registration and lifetime features are required.

✓ How to Achieve It

1. Create a console application.
2. Install the Autofac package.
3. Create interfaces and implementation classes.
4. Register services with ContainerBuilder.
5. Build the container and begin a lifetime scope.
6. Resolve the main service and execute its method.

Project Commands

```
dotnet new console -n AutofacDemo
cd AutofacDemo
dotnet add package Autofac
dotnet run
```

Complete Program.cs

```
using System;
using Autofac;

public interface IHome
{
    void ProvideShelter();
}

public interface ISchool
{
    void ProvideEducation();
}

public interface IHospital
{
    void ProvideTreatment();
}

public interface IPerson
{
    void PerformActivities();
}
```

```

public class Home : IHome
{
    public void ProvideShelter() =>
        Console.WriteLine("Home provides shelter.");
}

public class School : ISchool
{
    public void ProvideEducation() =>
        Console.WriteLine("School provides education.");
}

public class Hospital : IHospital
{
    public void ProvideTreatment() =>
        Console.WriteLine("Hospital provides treatment.");
}

public class Person : IPerson
{
    private readonly IHome _home;
    private readonly ISchool _school;
    private readonly IHospital _hospital;

    public Person(IHome home, ISchool school, IHospital hospital)
    {
        _home = home;
        _school = school;
        _hospital = hospital;
    }

    public void PerformActivities()
    {
        _home.ProvideShelter();
        _school.ProvideEducation();
        _hospital.ProvideTreatment();
    }
}

public static class Program
{
    public static void Main()
    {
        // KEY: Create Autofac registration builder.
        var builder = new ContainerBuilder();

        // KEY: Register implementations against interfaces.
        builder.RegisterType<Home>().As<IHome>();
        builder.RegisterType<School>().As<ISchool>();
        builder.RegisterType<Hospital>().As<IHospital>();
    }
}

```

```
builder.RegisterType<Person>().As<IPerson>();
```

```
// KEY: Build the Autofac container.
```

```
using IContainer container = builder.Build();
```

```
// KEY: Create a lifetime scope.
```

```
using ILifetimeScope scope = container.BeginLifetimeScope();
```

```
// KEY: Resolve IPerson; Autofac injects all dependencies.
```

```
IPerson person = scope.Resolve<IPerson>();
```

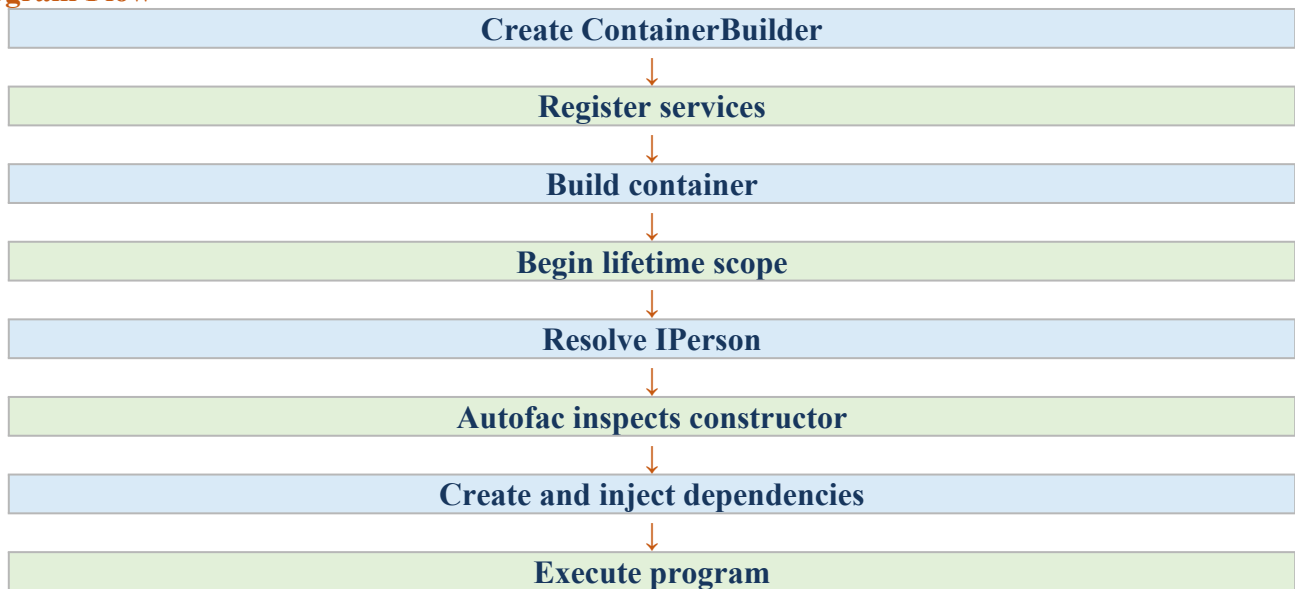
```
person.PerformActivities();
```

```
    }  
}
```

► Expected Output

- Home provides shelter.
- School provides education.
- Hospital provides treatment.

Program Flow



Types of Dependency Injection

• Short Note

- **Constructor Injection** is used for required dependencies.
- **Property Injection** is used for optional dependencies.
- **Method Injection** is used when only one method requires the dependency.
- Constructor Injection should be preferred whenever possible.

Complete Program Demonstrating All Three Types

```
using System;
```

```

public interface ILogger
{
    void Log(string message);
}

public class ConsoleLogger : ILogger
{
    public void Log(string message) => Console.WriteLine(message);
}

```

// 1. Constructor Injection

```

public class OrderService
{
    private readonly ILogger _logger;

    // KEY: Required dependency is supplied during object creation.
    public OrderService(ILogger logger)
    {
        _logger = logger;
    }

    public void CreateOrder() => _logger.Log("Order created.");
}

```

// 2. Property Injection

```

public class ReportService
{
    // KEY: Optional dependency is assigned after object creation.
    public ILogger? Logger { get; set; }

    public void GenerateReport()
    {
        Logger?.Log("Report generated.");
    }
}

//comment:

```

Dependency property

```
public ILogger? Logger { get; set; }
```

public

The property can be accessed and assigned from another class.

ILogger

This is the dependency type.

It normally represents a logging service that records messages, errors, or application activities.

?

ILogger?

The question mark means that the Logger property is **nullable**.

Therefore, it may contain:

- an object implementing ILogger, or
- null

This indicates that logging is an **optional dependency**.

Logger

This is the property name.

```
{ get; set; }
```

- get reads the property value.
- set assigns a value to the property.

// 3. Method Injection

```
public class BackupService
{
    // KEY: Dependency is required only for this method.
    public void CreateBackup(ILogger logger)
    {
        logger.Log("Backup created.");
    }
}

public static class Program
{
    public static void Main()
    {
        ILoggger logger = new ConsoleLogger();

        var orderService = new OrderService(logger);
        orderService.CreateOrder();

        var reportService = new ReportService
        {
            Logger = logger
        };
        reportService.GenerateReport();

        var backupService = new BackupService();
        backupService.CreateBackup(logger);
    }
}
```

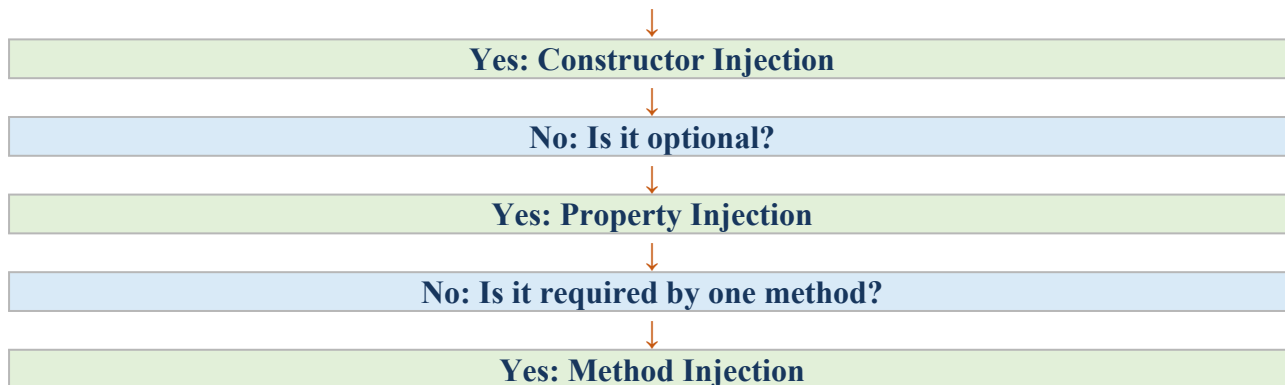
Type	Where Injected	Suitable Use	Main Risk
Constructor	Constructor	Required dependency	Few risks; preferred
Property	Public property	Optional dependency	Property may remain null
Method	Method parameter	One-method dependency	Must pass dependency each call

► Expected Output

- Order created.
- Report generated.
- Backup created.

Selection Flow

Is the dependency required by the entire class?



Service Lifetimes: Transient, Scoped, and Singleton

- **Short Note**

- A service lifetime determines **when a DI container creates a new object and how long it reuses that object.**
- **Transient:** a new object for every request.
- **Scoped:** one object within one scope.
- **Singleton:** one object for the complete application lifetime.

- **! Important Caution**

- A Singleton should be thread-safe.
- Do not store request-specific data in a Singleton.
- Do not inject a Scoped service directly into a Singleton.

Complete Lifetime Demonstration Program

```
using System;
using Microsoft.Extensions.DependencyInjection;

public interface IOperation
{
    Guid Id { get; }
}

public class Operation : IOperation
{
    public Guid Id { get; } = Guid.NewGuid();
}

public static class Program
{
    private static void Display(string title, IOperation first, IOperation second)
    {
        Console.WriteLine(title);
        Console.WriteLine($"First : {first.Id}");
        Console.WriteLine($"Second: {second.Id}");
        Console.WriteLine($"Same object: {ReferenceEquals(first, second)}");
        Console.WriteLine();
    }
}
```

```

public static void Main()
{
    TestTransient();
    TestScoped();
    TestSingleton();
}

private static void TestTransient()
{
    var services = new ServiceCollection();
    // KEY: A new object is returned every time.
    services.AddTransient<IOperation, Operation>();

    using var provider = services.BuildServiceProvider();
    var first = provider.GetRequiredService<IOperation>();
    var second = provider.GetRequiredService<IOperation>();
    Display("TRANSIENT", first, second);
}

private static void TestScoped()
{
    var services = new ServiceCollection();
    // KEY: The same object is reused within one scope.
    services.AddScoped<IOperation, Operation>();

    using var provider = services.BuildServiceProvider();
    using var scope1 = provider.CreateScope();
    var first = scope1.ServiceProvider.GetRequiredService<IOperation>();
    var second = scope1.ServiceProvider.GetRequiredService<IOperation>();
    Display("SCOPED - SCOPE 1", first, second);

    using var scope2 = provider.CreateScope();
    var third = scope2.ServiceProvider.GetRequiredService<IOperation>();
    Console.WriteLine($"SCOPED - NEW SCOPE: {third.Id}");
    Console.WriteLine();
}

private static void TestSingleton()
{
    var services = new ServiceCollection();
    // KEY: One object is reused throughout the provider lifetime.
    services.AddSingleton<IOperation, Operation>();

    using var provider = services.BuildServiceProvider();
    var first = provider.GetRequiredService<IOperation>();
    var second = provider.GetRequiredService<IOperation>();
    Display("SINGLETON", first, second);
}
}

```

► Expected Result Pattern

- **Transient:** different GUID values and `Same object: False`.
- **Scoped:** same GUID inside Scope 1, but a different GUID in Scope 2.
- **Singleton:** same GUID values and `Same object: True`.

Lifetime	Object Rule	Common Use
Transient	New object every resolve	Lightweight stateless services
Scoped	One object per request or scope	DbContext, repository, request service
Singleton	One object for application lifetime	Configuration, cache, shared stateless service

What Is an IoC Container?

An **IoC container**, or **Inversion of Control container**, is a framework that automatically creates objects, connects dependencies, injects them into classes, and manages their lifetime. Instead of a class creating its own dependency, the container creates and supplies it.

Main Responsibilities

An IoC container performs the following tasks:

- **Registers services**
- **Creates objects**
- **Resolves dependencies**
- **Injects dependencies**
- **Manages object lifetimes**
- **Disposes managed objects when required**

Simple Example

Suppose Person requires IHome.

```
public class Person
{
    private readonly IHome _home;

    public Person(IHome home)
    {
        _home = home;
    }
}
```

The container registration is:

```
builder.Services.AddTransient<IHome, Home>();
builder.Services.AddTransient<Person>();
```

This registration means:

When IHome is requested



Create and provide Home

How the IoC Container Works

When the application requests Person, the container checks its constructor:

```
public Person(IHome home)
```

The container sees that Person requires IHome.

It then checks the registration:

```
builder.Services.AddTransient<IHome, Home>();
```

The container automatically creates:

```
new Home()
```

and conceptually creates Person like this:

```
new Person(new Home());
```

The programmer does not need to create the dependency manually.

● Complete ASP.NET Core Example

Interface

```
public interface IHome
```

```
{  
    string ProvideShelter();  
}
```

Implementation

```
public class Home : IHome
```

```
{  
    public string ProvideShelter()  
    {  
        return "Home provides shelter.";  
    }  
}
```

Dependent Class

```
public class Person
```

```
{  
    private readonly IHome _home;  
  
    public Person(IHome home)  
    {  
        _home = home;  
    }  
  
    public string Live()  
    {  
        return _home.ProvideShelter();  
    }  
}
```

Registration in Program.cs

```
builder.Services.AddTransient<IHome, Home>();
```

```
builder.Services.AddTransient<Person>();
```

Controller

```
public class HomeController : Controller
{
    private readonly Person _person;

    public HomeController(Person person)
    {
        _person = person;
    }

    public IActionResult Index()
    {
        string message = _person.Live();

        return Content(message);
    }
}
```

Execution Flow

Browser requests Home page
↓
ASP.NET Core needs HomeController
↓
HomeController requires Person
↓
Person requires IHome
↓
IoC container checks registrations
↓
IHome is mapped to Home
↓
Container creates Home
↓
Home is injected into Person
↓
Person is injected into HomeController
↓
Controller action executes
↓
Response is returned

Common IoC Containers in .NET

IoC Container	Description
Microsoft Dependency	Built into ASP.NET Core

Injection	
Autofac	Popular third-party container with advanced features
Unity	General-purpose IoC container
Ninject	Lightweight dependency injection framework
Castle Windsor	Mature and flexible IoC container

● IoC Container vs Dependency Injection

Term	Meaning
IoC	General principle of transferring control
Dependency Injection	Technique used to supply dependencies
IoC Container	Tool that performs DI automatically

Service Lifetimes Managed by the Container

Transient

`builder.Services.AddTransient<IHome, Home>();`

A new object is created every time.

Scoped

`builder.Services.AddScoped<IHome, Home>();`

One object is created for each request or scope.

Singleton

`builder.Services.AddSingleton<IHome, Home>();`

One object is used throughout the application.

URL Routing in ASP.NET Core

1. Meaning of URL Routing

URL Routing is the process of matching an incoming URL with the correct controller and action method.

Example URL:

`/Students/Details/5`

Routing identifies:

Controller = StudentsController

Action = Details

ID = 5

2. Why Routing Is Used

Routing is used to:

- identify the correct controller;
- identify the correct action method;
- pass values from the URL;
- create clear and meaningful URLs;
- connect HTTP requests with application operations.

3. Types of Routing

You should study **two main types**:

1. Conventional Routing
2. Attribute Routing

3.1 Conventional Routing

Conventional routing is defined centrally in Program.cs.

Program

```
app.MapControllerRoute(  
    name: "default",  
    pattern: "{controller=Home}/{action=Index}/{id?}"  
);
```

Meaning

controller = Controller name

action = Action method name

id? = Optional route value

Example URL

/Students/Details/5

This matches:

```
public class StudentsController : Controller  
{  
    public IActionResult Details(int id)  
    {  
        return Content($"Student ID: {id}");  
    }  
}
```

Flow

URL received



Read controller name



Read action name



Read optional ID



Execute controller action

3.2 Attribute Routing

Attribute Routing defines the route directly above the controller or action.

Program

```
[Route("students")]  
public class StudentsController : Controller  
{
```

```

[HttpGet]
public IActionResult Index()
{
    return Content("Student list");
}

[HttpGet("details/{id:int}")]
public IActionResult Details(int id)
{
    return Content($"Student ID: {id}");
}
}

```

URLs

GET /students

GET /students/details/5

Major Point

[Route("students")]

defines the controller route.

[HttpGet("details/{id:int}")]

defines the action route and HTTP method.

4. Controller Route and Action Route

ASP.NET Core combines the controller route and action route.

Program

```

[ApiController]
[Route("api/students")]
public class StudentsController : ControllerBase
{
    [HttpGet("{id:int}")]
    public IActionResult GetById(int id)
    {
        return Ok(id);
    }
}

```

Complete URL

GET /api/students/10

The URL is formed from:

Controller route: /api/students

Action route: /{id:int}

Complete route: /api/students/10

● 5. Route Parameters

A route parameter receives a value directly from the URL.

Program

```
[HttpGet("students/{id}")]
public IActionResult GetStudent(int id)
{
    return Content($"Student ID: {id}");
}
```

URL

/students/10

Result

id = 10

You may write it explicitly as:

```
public IActionResult GetStudent(
    [FromRoute] int id)
```

6. Route Constraints

Route constraints restrict the type of value accepted by the route.

Program

```
[HttpGet("students/{id:int}")]
public IActionResult GetStudent(int id)
{
    return Ok(id);
}
```

Meaning

{id:int}

means that id must be an integer.

Valid URL

/students/10

Invalid URL

/students/ram

Common route constraints include:

:int

:bool

:decimal

:datetime

:guid

For your current level, understanding :int is usually sufficient.

7. Query String Is Not Part of the Route Path

A query string appears after the ? symbol.

Program

```
[HttpGet("search")]
public IActionResult Search(
    [FromQuery] string name)
{
    return Ok(name);
}
```

```
}
```

URL

/api/students/search?name=Ram

Breakdown:

/api/students/search = Route

?name=Ram = Query string

Key Difference

Route parameter	Query string
Part of URL path	Written after ?
Usually identifies a resource	Usually filters or searches
/students/10	/students?name=Ram

8. HTTP Method Attributes

Routing also checks the HTTP method.

Attribute	Purpose
[HttpGet]	Read data
[HttpPost]	Create data
[HttpPut]	Update complete data
[HttpPatch]	Update partial data
[HttpDelete]	Delete data

Example

```
[Route("api/students")]
public class StudentsController : ControllerBase
{
    [HttpGet]
    public IActionResult GetAll()
    {
        return Ok();
    }

    [HttpPost]
    public IActionResult Create()
    {
        return Ok();
    }
}
```

Both actions use the same route:

/api/students

But ASP.NET Core selects the action according to the HTTP method:

GET /api/students → GetAll()

POST /api/students → Create()

● 9. Enabling Routing in Program.cs

For API Controllers

```
builder.Services.AddControllers();
```

```
var app = builder.Build();
```

```
app.MapControllers();
```

```
app.Run();
```

Important Line

```
app.MapControllers();
```

This enables routes defined through attributes such as:

```
[Route]
```

```
[HttpGet]
```

```
[HttpPost]
```

For MVC Controllers

```
builder.Services.AddControllersWithViews();
```

```
var app = builder.Build();
```

```
app.MapControllerRoute(  
    name: "default",  
    pattern: "{controller=Home}/{action=Index}/{id?}"  
);
```

```
app.Run();
```

This enables conventional MVC routing.

10. Short Applicable Program

Program.cs

```
var builder = WebApplication.CreateBuilder(args);
```

```
builder.Services.AddControllers();
```

```
var app = builder.Build();
```

```
app.MapControllers();
```

```
app.Run();
```

StudentsController.cs

```
using Microsoft.AspNetCore.Mvc;
```

```
[ApiController]
```

```

[Route("api/students")]
public class StudentsController : ControllerBase
{
    [HttpGet]
    public IActionResult GetAll()
    {
        return Ok("All students");
    }

    [HttpGet("{id:int}")]
    public IActionResult GetById(int id)
    {
        return Ok($"Student ID: {id}");
    }

    [HttpGet("search")]
    public IActionResult Search(
        [FromQuery] string name)
    {
        return Ok($"Student Name: {name}");
    }

    [HttpPost]
    public IActionResult Create()
    {
        return Ok("Student created");
    }
}

```

Available URLs

GET /api/students

GET /api/students/10

GET /api/students/search?name=Ram

POST /api/students

Routing Execution Flow

Client sends URL and HTTP method



ASP.NET Core routing system receives request



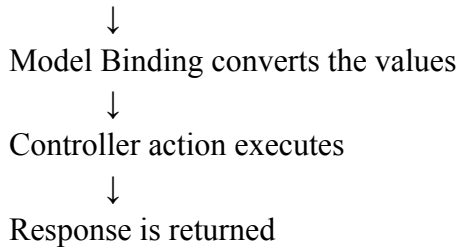
Controller route is matched



Action route and HTTP method are matched



Route values are extracted



Models, Model Binding, and Validation in ASP.NET Core

1. Model

A **Model** is a C# class that represents application data.

```
public class StudentRequest
{
    public string Name { get; set; } = string.Empty;

    public int Age { get; set; }

    public string Email { get; set; } = string.Empty;
}
```

Essential Points

- StudentRequest is the model class.
- Name, Age, and Email are model properties.
- The model stores data received from the client.
- It can be used in MVC forms or Web API requests.

2. Model Binding

● Definition

Model Binding automatically converts HTTP request data into C# parameters or model objects.

ASP.NET Core can receive data from:

- Route
- Query string
- HTML form
- JSON body
- HTTP header

Route Binding

Program

```
[HttpGet("students/{id}")]
public IActionResult GetStudent(
    [FromRoute] int id)
{
    return Content($"Student ID: {id}");
}
```

Request

/students/10

Output

Student ID: 10

Key Point

[FromRoute] int id

reads the value from the URL route.

Query String Binding

Program

[HttpGet("search")]

public IActionResult Search(

[FromQuery] string name)

{

return Content(\$"Student Name: {name}");

}

Request

/search?name=Ram

Output

Student Name: Ram

Key Point

[FromQuery] string name

reads the value from the query string.

JSON Body Binding

Program

[HttpPost]

public IActionResult Create(

[FromBody] StudentRequest request)

{

return Ok(request);

}

JSON Request

{

"name": "Ram",

"age": 20,

"email": "ram@example.com"

}

Key Point

[FromBody] StudentRequest request

converts JSON data into a C# object.

3. Model Validation

● Definition

Model Validation checks whether the received data is correct, complete, and acceptable.

Validation rules are normally added using Data Annotation attributes.

Common Validation Attributes

Attribute	Purpose
[Required]	Field must contain a value
[Range]	Number must be within a limit
[EmailAddress]	Value must be a valid email
[StringLength]	Controls text length
[Compare]	Compares two properties

4. Short Applicable Program

Model: StudentRequest.cs

using System.ComponentModel.DataAnnotations;

```
public class StudentRequest
{
    [Required(ErrorMessage = "Name is required.")]
    [StringLength(
        50,
        MinimumLength = 3,
        ErrorMessage = "Name must contain 3 to 50 characters."
    )]
    public string Name { get; set; } = string.Empty;

    [Range(
        1,
        100,
        ErrorMessage = "Age must be between 1 and 100."
    )]
    public int Age { get; set; }

    [Required(ErrorMessage = "Email is required.")]
    [EmailAddress(ErrorMessage = "Enter a valid email.")]
    public string Email { get; set; } = string.Empty;
}
```

Controller: StudentsController.cs

using Microsoft.AspNetCore.Mvc;

```
[ApiController]
[Route("api/students")]
public class StudentsController : ControllerBase
```

```

{
    [HttpPost]
    public IActionResult Create(
        [FromBody] StudentRequest request)
    {
        return Ok(new
        {
            message = "Student created successfully.",
            student = request
        });
    }
}

```

Program Configuration: Program.cs

```
var builder = WebApplication.CreateBuilder(args);
```

```
builder.Services.AddControllers();
```

```
var app = builder.Build();
```

```
app.MapControllers();
```

```
app.Run();
```

5. How to Demonstrate the Program

Step 1: Run the application

```
dotnet run
```

Step 2: Send a valid JSON request

```

{
    "name": "Ram",
    "age": 20,
    "email": "ram@example.com"
}

```

Expected Result

```

{
    "message": "Student created successfully.",
    "student": {
        "name": "Ram",
        "age": 20,
        "email": "ram@example.com"
    }
}

```

Step 3: Send invalid data

```
{
  "name": "",
  "age": 150,
  "email": "wrong-email"
}
```

● Expected Validation Errors

- Name is required.
- Age must be between 1 and 100.
- Email format is invalid.
- ASP.NET Core returns 400 Bad Request.

6. Execution Flow

Client sends HTTP request



Model Binding reads request data



StudentRequest object is created



Validation attributes are checked



Is the model valid?



Yes



No



Action runs



400 Bad Request



Success response

Rendering HTML with Views in ASP.NET Core MVC

1. What Is a View?

A **View** is a .cshtml file that generates HTML for the browser.

The Controller sends data to the View, and the Razor View Engine converts the View into normal HTML.

Essential Flow

Browser Request



Controller Action



Model Data



Razor View (.cshtml)



Generated HTML



Browser

Simple Controller

```
public class HomeController : Controller
{
    public IActionResult Index()
    {
        return View();
    }
}
```

ASP.NET Core searches for:

Views/Home/Index.cshtml

Simple View

```
<h1>Welcome to ASP.NET Core MVC</h1>
```

```
<p>This HTML is generated by a Razor View.</p>
```

2. Passing a Model to a View

A Controller can pass model data to a View.

Model

```
public class Product
{
    public string Name { get; set; } = string.Empty;

    public decimal Price { get; set; }
}
```

Controller

```
public IActionResult Details()
{
    var product = new Product
    {
        Name = "Laptop",
        Price = 75000
    };

    return View(product);
}
```

View

```
@model Product
```

```
<h1>@Model.Name</h1>
```

```
<p>Price: @Model.Price</p>
```

Major Points

- return View(product) sends the model to the View.
- @model Product declares the model type.
- @Model.Name displays the product name.
- @Model.Price displays the product price.

3. What Is Razor Syntax?

Razor Syntax is used to write C# code inside HTML.

Razor uses the @ symbol.

3.1 Display a Variable

```
@{  
    string name = "Ram";  
}
```

```
<h2>@name</h2>
```

Output

Ram

3.2 Razor Code Block

```
@{  
    int age = 20;  
    string message = "Student age is " + age;  
}
```

```
<p>@message</p>
```

3.3 Conditional Statement

```
@{  
    bool isAvailable = true;  
}  
  
@if (isAvailable)  
{  
    <p>Product is available.</p>  
}  
else  
{  
    <p>Product is not available.</p>  
}
```

3.4 Loop

```
@{  
    string[] subjects =  
    {  
        "C#",  
        "ASP.NET Core",  
        "SQL"  
    };  
}
```

```

<ul>
  @foreach (string subject in subjects)
  {
    <li>@subject</li>
  }
</ul>

```

3.5 Razor Comment

@* This comment is not sent to the browser. *@

Important Point

Razor comments are processed on the server and do not appear in the browser source.

4. Important Razor Directives

@model

Defines the View model type.

@model Product

@using

Imports a namespace.

@using MyMvcApp.Models

@inject

Injects a registered service into the View.

@inject IDateTimeService DateTimeService

```
<p>@DateTimeService.Now</p>
```

@section

Adds page-specific CSS or JavaScript to a Layout.

@section Scripts

```

{
  <script>
    console.log("Page loaded");
  </script>
}

```

● 5. What Is a Layout?

A **Layout** contains common HTML used by several pages.

It commonly contains:

- Header
- Navigation
- Main content area
- Footer
- CSS and JavaScript references

_Layout.cshtml

```
<!DOCTYPE html>
```

```

<html>
<head>
  <title>@ViewData["Title"]</title>
</head>

<body>
  <header>
    <h2>My Application</h2>
  </header>

  <main>
    @RenderBody()
  </main>

  <footer>
    <p>Copyright @DateTime.Now.Year</p>
  </footer>
</body>
</html>

```

Major Point

@RenderBody()

is the location where the content of each individual View is displayed.

6. What Are Tag Helpers?

Tag Helpers are server-side features that simplify HTML generation in Razor Views.

They use attributes beginning with asp-.

Tag Helpers help generate:

- Links
- Forms
- Input elements
- Labels
- Validation messages
- Drop-down lists
- Images
- Partial Views

7. Enable Tag Helpers

In _ViewImports.cshtml:

```
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

This makes ASP.NET Core built-in Tag Helpers available in all Views.

8. Important Tag Helpers

8.1 Anchor Tag Helper

Creates a link.

```
<a asp-controller="Products"
    asp-action="Details"
    asp-route-id="10">
    View Details
</a>
```

Generated HTML may be:

```
<a href="/Products/Details/10">
    View Details
</a>
```

Important Attributes

- asp-controller: Controller name
- asp-action: Action name
- asp-route-id: Route value

8.2 Form Tag Helper

Creates a form action URL.

```
<form asp-controller="Products"
      asp-action="Create"
      method="post">

    <button type="submit">Save</button>
</form>
```

8.3 Input Tag Helper

Connects an input with a model property.

```
<input asp-for="Name" />
```

It can generate:

```
<input type="text" id="Name" name="Name" />
```

8.4 Label Tag Helper

```
<label asp-for="Name"></label>
```

It generates a label connected to the property.

8.5 Validation Message Tag Helper

```
<span asp-validation-for="Name"></span>
```

Displays the validation error for Name.

8.6 Validation Summary Tag Helper

```
<div asp-validation-summary="All"></div>
```

Displays all validation errors.

8.7 Textarea Tag Helper

```
<textarea asp-for="Description"
           rows="5">
```

</textarea>

8.8 Select Tag Helper

```
<select asp-for="CategoryId"
        asp-items="Model.Categories">

    <option value="">Select category</option>
</select>
```

8.9 Partial Tag Helper

```
<partial name="_ProductCard"
        model="product" />
```

It renders reusable View content.

● 9. Razor Syntax vs Tag Helpers

Razor Syntax	Tag Helpers
Uses @	Uses asp- attributes
Executes C# logic	Generates or modifies HTML
Used for conditions and loops	Used for links, forms, and inputs
@Model.Name	<input asp-for="Name">
@foreach	<partial name="...">

Combined Example

```
@foreach (var product in Model)
{
    <a asp-controller="Products"
      asp-action="Details"
      asp-route-id="@product.Id">

        @product.Name
    </a>
}
```

Here:

- @foreach is Razor Syntax.
- @product.Name is a Razor expression.
- asp-controller is a Tag Helper.
- asp-action is a Tag Helper.
- asp-route-id is a Tag Helper.

10. Short Applicable Program

Model: ProductRequest.cs

```
using System.ComponentModel.DataAnnotations;
```

```
public class ProductRequest
```

```

{
    [Required(ErrorMessage = "Product name is required.")]
    public string Name { get; set; } = string.Empty;

    [Range(
        1,
        1000000,
        ErrorMessage = "Price must be greater than zero."
    )]
    public decimal Price { get; set; }
}

```

Controller: ProductsController.cs

```
using Microsoft.AspNetCore.Mvc;
```

```

public class ProductsController : Controller
{
    [HttpGet]
    public IActionResult Create()
    {
        return View();
    }

    [HttpPost]
    public IActionResult Create(ProductRequest request)
    {
        if (!ModelState.IsValid)
        {
            return View(request);
        }

        return RedirectToAction(nameof(Success));
    }

    public IActionResult Success()
    {
        return Content("Product created successfully.");
    }
}

```

View: Views/Products/Create.cshtml

```
@model ProductRequest
```

```
@{
```

```

    ViewData["Title"] = "Create Product";
}

<h1 class="text-primary">Create Product</h1>

<form asp-controller="Products"
      asp-action="Create"
      method="post">

    <div asp-validation-summary="All"
        class="text-danger">
    </div>

    <div>
        <label asp-for="Name"></label>

        <input asp-for="Name"
            class="form-control" />

        <span asp-validation-for="Name"
            class="text-danger">
        </span>
    </div>

    <div>
        <label asp-for="Price"></label>

        <input asp-for="Price"
            class="form-control" />

        <span asp-validation-for="Price"
            class="text-danger">
        </span>
    </div>

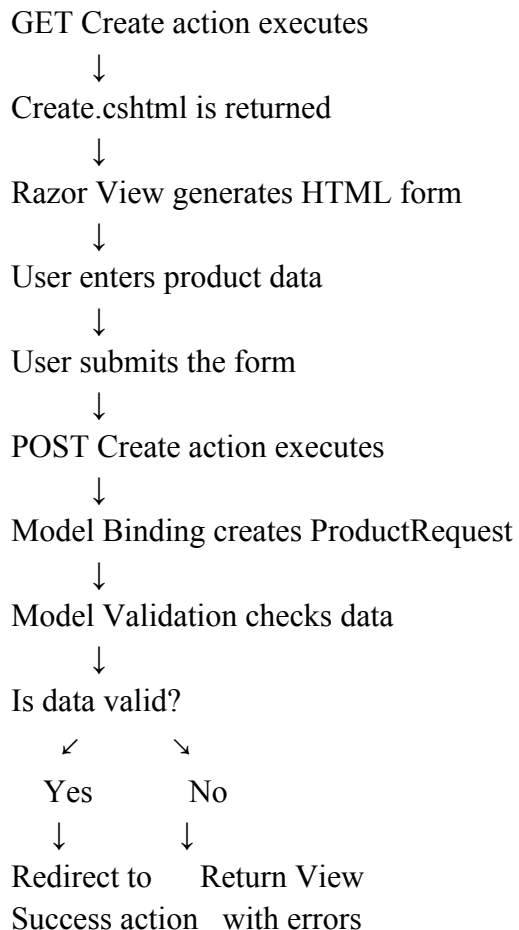
    <button type="submit"
        class="btn btn-success">
        Save
    </button>
</form>

```

11. Program Execution Flow

Browser requests Create page





Web API Applications: API Controllers and JSON

1. What Is a Web API?

A **Web API** allows different applications to communicate through HTTP.

A Web API may be used by:

- Mobile applications
- React, Angular, or Vue applications
- Desktop applications
- Other servers
- IoT devices

Unlike an MVC View, a Web API normally returns **JSON data**, not HTML.

Basic Flow

Client

↓ HTTP Request

API Controller

↓

Business Logic or Service

↓

Database or Data Source

↓

JSON Response

2. What Is an API Controller?

An **API Controller** is a C# class that receives and processes HTTP requests.

It normally:

- inherits from ControllerBase;
- uses [ApiController];
- uses [Route];
- contains action methods;
- returns JSON and HTTP status codes.

Basic Structure

using Microsoft.AspNetCore.Mvc;

```
[ApiController]
[Route("api/[controller]")]
public class ProductsController : ControllerBase
{
}
```

Essential Points

- ControllerBase provides methods such as Ok(), NotFound(), and BadRequest().
- [ApiController] enables API-specific behaviour.
- [Route("api/[controller]")] defines the URL.
- [controller] is replaced by the controller name without the word Controller.

For ProductsController, the base URL becomes:

/api/products

3. Program Configuration

Program.cs

```
var builder = WebApplication.CreateBuilder(args);
```

```
// Register API controller services.
```

```
builder.Services.AddControllers();
```

```
var app = builder.Build();
```

```
app.UseHttpsRedirection();
```

```
// Enable attribute-routed controllers.
```

```
app.MapControllers();
```

```
app.Run();
```

Major Lines

```
builder.Services.AddControllers();
```

Registers controller-related services.

```
app.MapControllers();
```

Activates routes defined by:

- [Route]
- [HttpGet]
- [HttpPost]
- [HttpPut]
- [HttpDelete]

4. JSON

JSON stands for JavaScript Object Notation.

It is a lightweight format used to exchange data between a client and a server.

JSON Example

```
{
  "id": 1,
  "name": "Laptop",
  "price": 75000
}
```

Important Terms

- **Serialization** converts a C# object into JSON.
- **Deserialization** converts JSON into a C# object.

Flow

C# Object

↓ Serialization

JSON Response

JSON Request

↓ Deserialization

C# Object

● 5. Product Model

Models/Product.cs

namespace ProductApi.Models;

public class Product

```
{
    public int Id { get; set; }

    public string Name { get; set; } = string.Empty;

    public decimal Price { get; set; }

    public int Quantity { get; set; }

    public bool IsAvailable { get; set; }
}
```

Essential Point

The Product model represents the data returned by the API.

6. Request Model with Validation

Models/CreateProductRequest.cs

using System.ComponentModel.DataAnnotations;

namespace ProductApi.Models;

```
public class CreateProductRequest
{
    [Required(ErrorMessage = "Product name is required.")]
    public string Name { get; set; } = string.Empty;

    [Range(
        1,
        1000000,
        ErrorMessage = "Price must be greater than zero."
    )]
    public decimal Price { get; set; }

    [Range(
        0,
        10000,
        ErrorMessage = "Quantity cannot be negative."
    )]
    public int Quantity { get; set; }
}
```

Major Point

[ApiController] normally returns **400 Bad Request automatically** when validation fails.

7. Complete API Controller

Controllers/ProductsController.cs

using Microsoft.AspNetCore.Mvc;

using ProductApi.Models;

namespace ProductApi.Controllers;

```
[ApiController]
[Route("api/[controller]")]
public class ProductsController : ControllerBase
{
    private static readonly List<Product> Products =
    [
        new Product
        {

```

```

        Id = 1,
        Name = "Laptop",
        Price = 75000,
        Quantity = 5,
        IsAvailable = true
    },

    new Product
    {
        Id = 2,
        Name = "Mobile",
        Price = 35000,
        Quantity = 0,
        IsAvailable = false
    }
];

// GET: /api/products
[HttpGet]
public ActionResult<IEnumerable<Product>> GetAll()
{
    return Ok(Products);
}

// GET: /api/products/1
[HttpGet("{id:int}")]
public ActionResult<Product> GetById(int id)
{
    Product? product =
        Products.FirstOrDefault(p => p.Id == id);

    if (product is null)
    {
        return NotFound(new
        {
            message = "Product not found."
        });
    }

    return Ok(product);
}

// POST: /api/products
[HttpPost]

```

```

public ActionResult<Product> Create(
    [FromBody] CreateProductRequest request)
{
    int newId = Products.Count == 0
        ? 1
        : Products.Max(p => p.Id) + 1;

    var product = new Product
    {
        Id = newId,
        Name = request.Name,
        Price = request.Price,
        Quantity = request.Quantity,
        IsAvailable = request.Quantity > 0
    };

    Products.Add(product);

    return CreatedAtAction(
        nameof(GetById),
        new { id = product.Id },
        product
    );
}

// DELETE: /api/products/1
[HttpDelete("{id:int}")]
public IActionResult Delete(int id)
{
    Product? product =
        Products.FirstOrDefault(p => p.Id == id);

    if (product is null)
    {
        return NotFound();
    }

    Products.Remove(product);

    return NoContent();
}
}

```

● 8. HTTP Methods and URLs

HTTP Method	Purpose	Example URL
GET	Read all data	/api/products
GET	Read one item	/api/products/1
POST	Create new data	/api/products
PUT	Update complete data	/api/products/1
PATCH	Update partial data	/api/products/1
DELETE	Delete data	/api/products/1

9. GET Request

Action

```
[HttpGet]
public ActionResult<IEnumerable<Product>> GetAll()
{
    return Ok(Products);
}
```

Request

GET /api/products

Response

```
[
  {
    "id": 1,
    "name": "Laptop",
    "price": 75000,
    "quantity": 5,
    "isAvailable": true
  },
  {
    "id": 2,
    "name": "Mobile",
    "price": 35000,
    "quantity": 0,
    "isAvailable": false
  }
]
```

Major Point

return Ok(Products);
returns:
200 OK
with JSON data.

10. GET by ID

Action

```
[HttpGet("{id:int}")]
public ActionResult<Product> GetById(int id)
```

Request

GET /api/products/1

Route Meaning

/api/products = Controller route

/1 = Route parameter

GET = HTTP method

If the product is found:

200 OK

If it is not found:

404 Not Found

11. POST Request

Action

```
[HttpPost]
public ActionResult<Product> Create(
    [FromBody] CreateProductRequest request)
```

Major Point

[FromBody]

converts JSON request data into a C# object.

Request

POST /api/products

Content-Type: application/json

JSON Body

```
{
  "name": "Keyboard",
  "price": 2500,
  "quantity": 20
}
```

Successful Response

201 Created

```
{
  "id": 3,
  "name": "Keyboard",
  "price": 2500,
  "quantity": 20,
  "isAvailable": true
}
```

Important Line

```
return CreatedAtAction(
    nameof(GetById),
    new { id = product.Id },
    product)
```

);

This returns:

- 201 Created
- created product data
- a URL for retrieving the new product

● 12. Invalid Request

JSON Request

```
{  
  "name": "",  
  "price": -500,  
  "quantity": -2  
}
```

Because validation fails, ASP.NET Core returns:

400 Bad Request

Possible response:

```
{  
  "errors": {  
    "Name": [  
      "Product name is required."  
    ],  
    "Price": [  
      "Price must be greater than zero."  
    ],  
    "Quantity": [  
      "Quantity cannot be negative."  
    ]  
  }  
}
```

● 13. Common HTTP Status Codes

Status Code	Meaning	ASP.NET Core Method
200	Request succeeded	Ok()
201	Resource created	Created() or CreatedAtAction()
204	Success with no response body	NoContent()
400	Invalid request	BadRequest()
401	Authentication required	Unauthorized()
403	Permission denied	Forbid()
404	Resource not found	NotFound()
409	Data conflict	Conflict()
500	Server error	Problem()

14. API Controller vs MVC Controller

API Controller	MVC Controller
Inherits from ControllerBase	Inherits from Controller
Returns JSON	Returns HTML Views
Uses Ok(data)	Uses View(model)
Used by mobile or frontend clients	Used by browser pages
Commonly uses attribute routing	May use conventional routing

15. Complete Request-Response Flow

Client sends HTTP request



Routing selects API Controller



HTTP method selects Action



Model Binding reads request data



JSON is converted into a C# object



Model Validation checks data



Controller Action executes



Business logic processes data



Action Result is selected



C# result is serialized into JSON



HTTP status and JSON are returned